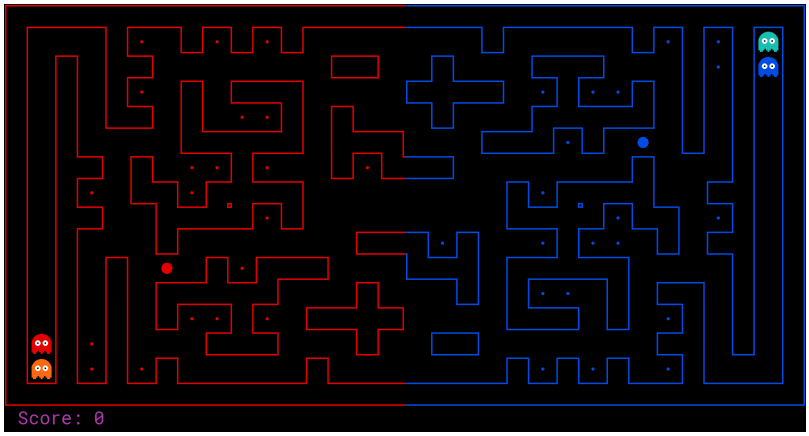
Project 4: Pacman Capture the Flag



Enough of defense,
Onto enemy terrain.
Capture all their food!

Introduction

The final project involves a multi-player capture-the-flag variant of Pacman, where agents control both Pacman and ghosts in coordinated team-based strategies. Your team will try to eat the food on the far side of the map, while defending the food on your home side.

The project will consist of three phases:

1. The first phase will consist of finding a team (of no more than three people) and an initial test of the tournament. Students must have a team and at **minimum** must submit a DummyAgent for this phase.
2. The second phase will consist of daily round-robin style tournaments. During this phase students should improve their agent, while looking for weaknesses in their opponents' submissions.
3. The final phase will be a final round-robin style tournament, in which students show their best. This is students' final chance to shine, winner takes all!

All of the deadlines for these events will exist on **Canvas**. Make sure you check early, **do not** miss a deadline!

We will evaluate your submissions based on a short written report (2-3 pages) on your modeling of the problem and agent design, as well as your performance against your classmates in tournament play.

Submission

To enter into the nightly tournaments, your team's agents and all relevant functions must be defined in pacai.student.myTeam.

Every team must have a unique name, consisting only of ASCII letters and digits (any other characters, including whitespace, will be ignored). Fill in your team name and list all team members in this form. Please access this document using your UCSC account as this is intended to be shared with only with valid UCSC accounts. As shown in the Google doc, you will state your chosen team name, motto, and members. In every submission to the autograder (linked below), you must include a file `name.txt` in which you will write only your unique team name. **Do not** include other extraneous text in this file. Only your team name will be displayed to the rest of the class.

This assignment should be submitted with the filename **`solution.zip`** HERE. Please use the command line `zip` tool to make sure your submission gets zipped correctly:

```
zip -j solution.zip pacai/student/myTeam.py pacai/student/name.txt
```

For these submissions, unzip should directly yield the source files and **not** a directory named "solution", "pN", or "pacai". Note that this is counter to standard conventions when sending a zip file to a human, but

easier for the autograder. Since there isn't a "correct" solution to this assignment, the autograder will report whether your agent beat the provided pacai.core.baselineTeam agent within the contest rules.

For your **final paper submission**, one team member will upload a file named `[your team name].pdf` that contains your write-up to Canvas. Please make sure that this document contains the names of all members of your team clearly stated at the top.

Evaluation

The contest will count as your final project, worth 40 points. 20 of these points will be the result of a written report you submit with your agent describing your approach. The remaining 20 points will be awarded based on your performance in the final contest.

The written report should be 2-3 pages **(no more)**. Through this report we expect you to demonstrate your ability to constructively solve AI problems by identifying:

- The fundamental problems you are trying to solve.
- How you modeled these problems.
- Your representations of the problems.
- The computational strategy used to solve each problem.
- Algorithmic choices you made in your implementation.
- Any obstacles you encountered while solving the problem.
- Evaluation of your agent.
- Lessons learned during the project.

A portion of your grade will be based on performance against the following staff agents:

- staff_baseline
- staff_SlugTrap
- staff_SomeSlug
- staff_SlugBrain

The performance-based portion of your grade will be awarded as follows:

- If you lose to the dummy agent, **zero points** will be awarded for this section.
- 10 points for beating the staff_baseline agent.
- +5 points for beating one additional staff agent. (15 points for beating the staff_baseline agent and one of the additional staff agents).
- +5 points for beating 2 staff agents (in addition to staff_baseline), OR.
    - +1 points for being in the top 50%.
    - +2 points for being in the top 40%.
    - +3 points for being in the top 30%.
    - +4 points for being in the top 20%.
    - +5 points for being in the top 10%.
- +1 Extra Credit point for being the number one team.

How we compute the percentiles based on the ranking of the teams is described below in Contest Details.

Academic Dishonesty

We will be checking your code against other submissions in the class for logical redundancy. If you copy someone else's code and submit it with minor changes, we will know. These cheat detectors are quite hard to fool, so please don't try. We trust you all to submit your own work only; *please* don't let us down. If you do, we will pursue the strongest consequences available to us.

Getting Help

You are not alone! If you find yourself stuck on something, contact the course staff for help. Office hours, section, and Piazza are there for your support; please use them. If you can't make our office hours, let us know and we will schedule more. We want these projects to be rewarding and instructional, not frustrating and demoralizing. But, we don't know when or how to help unless you ask. One more piece of advice: if you don't know what a variable does or what kind of values it takes, print it out.

Running Code on Your Local Machine

In order to run the Pac-Man code on your local machine, you must have Tk, python >= 3.5, and pip installed. Finally you want to install Pacai's package requirements listed in the `requirements.txt` file in

the project's root directory:

```
pip3 install --user -r requirements.txt
```

For the next set of instructions, simply follow the steps listed below depending on your OS.

Linux

Install the Python binding for the Tk package, usually called something like `python3-tk`. On Ubuntu you can use the following command:

```
sudo apt-get install python3-tk
```

Mac OS X

The required additional components (Tk and Python3 Tk bindings) are typically bundled with the Python3 package.

Windows

There are two separate methods for getting Pacai to work on Windows. One is using the Windows Subsystem for Linux (WSL) and the other is using Git Bash. We will first talk about the WSL.

Windows Subsystem for Linux (WSL)

1. First, make sure you have the WSL installed on your local machine. You can follow this installation guide. You are allowed to pick any distribution, however we will be providing specific instructions for Ubuntu. Other distributions will have similar steps.

2. Next, you want to make sure you have an X server running. This allows the WSL to launch graphical applications. We recommend using VcXsrv and following this installation guide. Make sure your X server is running whenever you want to run Pacai with graphics.

3. Now launch your WSL. You can do this by typing "Windows Subsystem for Linux" in your start menu and clicking on the icon.

4. Next, you must configure your bash inside the WSL to use the X server from step #2. To do this, use the following commands:

```
echo "export DISPLAY=localhost:0.0" >> ~/.bashrc
source ~/.bashrc
```

This sets your `DISPLAY` environmental variable in your bashrc and then reloads your bashrc.

5. Let's make sure you have all the required packages:

```
sudo apt-get update
sudo apt-get install python3 python3-pip python3-tk x11-apps
```

7. In order to test that you have the graphics set up correctly, you can use the `xeyes` command:

```
xeyes
```

8. Finally, clone the Pacai repository and install Pacai's package requirements:

```
pip3 install --user -r requirements.txt
```

Git Bash

1. Install Git Bash, you can follow this installation guide.

2. Now launch your Git Bash. You can do this by typing "Git Bash" in your start menu and clicking on the icon.

3. If you are using a newer version of Windows, there may be a conflict with the version of Python 3 installed from the Windows Store. This conflict will cause a permission denied error when running Python 3 in Git Bash. To avoid this error, you will want to disable the Windows Store version of Python. To do this, type "manage app execution aliases" into your start menu and click on the icon. Within the app execution aliases, disable both `python.exe` and `python3.exe`.

4. You may also want to create an alias for `python3`. All the commands in the instructions use `python3`, but Git Bash just refers to the executable as `python`. To create the alias, you can use the following commands:

```
echo "alias python3=python" >> ~/.bash_profile
source ~/.bash_profile
```

5. Finally, clone the Pacai repository and install Pacai's package requirements:

```
pip3 install --user -r requirements.txt
```

Pacman Capture the Flag

Layout: The Pacman map is now divided into two halves: red (left) and blue (right). Red agents (which all have even indices) must defend the red food while trying to eat the blue food. When on the red side, a red agent is a ghost. When crossing into enemy territory, the agent becomes a Pacman.

Scoring: When a Pacman eats a food dot, the food is permanently removed and one point is scored for that Pacman's team. Red team scores are positive, while Blue team scores are negative.

Eating Pacman: When a Pacman is eaten by an opposing ghost, the Pacman returns to its starting position (as a ghost). No points are awarded for eating an opponent.

Power capsules: If Pacman eats a power capsule, agents on the opposing team become "scared" for the next 40 moves, or until they are eaten and respawn, whichever comes sooner. Agents that are "scared" are susceptible while in the form of ghosts (i.e. while on their own team's side) to being eaten by Pacman. Specifically, if Pacman collides with a "scared" ghost, Pacman is unaffected and the ghost respawns at its starting position (no longer in the "scared" state).

Winning: A game ends when one team eats all but two of the opponents' dots. Games are also limited to 1200 agent moves (300 moves per each of the four agents). If this move limit is reached, whichever team has eaten the most food wins. If the score is zero (i.e., tied) this is recorded as a tie game.

Computation Time: We will run your submissions on a VM server. Each move which does not return within one second will incur a warning. After three warnings, or any single move taking more than 3 seconds, the game is forfeit. There will be an initial start-up allowance of 15 seconds (use the pacai.agents.capture.capture.CaptureAgent.registerInitialState function). If you agent times out or otherwise throws an exception, an error message will be present in the log files, which you can download from the results page (see below).

Getting Started

By default, you can run a game with the simple pacai.core.baselineTeam that the staff has provided:

```
python3 -m pacai.bin.capture
```

A wealth of options are available to you:

```
python3 -m pacai.bin.capture --help
```

There are four slots for agents, where agents 0 and 2 are always on the red team, and agents 1 and 3 are on the blue team. See the section on designing agents for a description of the agents invoked above. The only team that we provide is the baselineTeam. It is chosen by default as both the red and blue team, but as an example of how to choose teams:

```
python3 -m pacai.bin.capture --red pacai.core.baselineTeam --blue pacai.core.baselineTeam
```

which specifies that the red team `--red` and the blue team `--blue` are both created from baselineTeam. To control one of the four agents with the keyboard, pass the appropriate option:

```
python3 -m pacai.bin.capture --keys0
```

The arrow keys control your character, which will change from ghost to Pacman when crossing the center line.

Layouts

By default, all games are run on the `defaultcapture` layout. To test your agent on other layouts, use the `--layout` option. In particular, you can generate random layouts by specifying `RANDOM[seed]`. For example, `--layout RANDOM140` will use a map randomly generated with seed 140.

Game Types

You can play the game in two ways: local games and nightly tournaments. Local games (described above) allow you to test your agents against the baseline teams we provide and are intended for use in development.

Official Tournaments

The round-robin contests will be run using nightly automated tournaments on the course server, with the final tournament deciding the final contest outcomes. See the submission instructions for details of how to enter a team into the tournaments. Tournaments are run every night (refer to Canvas for nightly cut off) and include all teams that have been submitted (either earlier in the day or on a previous day) as of the start of the tournament. Currently, each team plays every other team 15 times in one match.

The layouts used in the tournament will be drawn from both the default layout (5 games), as well as randomly generated layouts (10 games). All layouts are symmetric, and the team that moves first is randomly chosen. The results for a nightly tournaments can be found here (TBA), where you can view overall rankings and scores for each match. You can also download replays, the layouts used, and the stdout / stderr logs for each agent.

Designing Agents

Unlike the other projects, an agent now has the more complex job of trading off offense versus defense and effectively functioning as both a ghost and a Pacman in a team setting. Furthermore, the limited information provided to your agent will likely necessitate some probabilistic tracking. Finally, the added time limit of computation introduces new challenges.

Baseline Team: To kickstart your agent design, we have provided you with a team of two baseline agents, defined in pacai.core.baselineTeam. They are both quite bad. The pacai.agents.capture.offense.OffensiveReflexAgent moves toward the closest food on the opposing side. The pacai.agents.capture.defense.DefensiveReflexAgent wanders around on its own side and tries to chase down invaders it happens to see.

File naming: For the purpose of testing or running games locally, you can define a team of agents in any arbitrarily-named python3 file. When submitting to the nightly tournament, however, you must define your agents in pacai.student.myTeam (and you must also create a `name.txt` file that specifies your team name).

Interface: The pacai.bin.capture.CaptureGameState should look familiar, but contains new methods like getRedFood, which gets a grid of food on the red side (note that the grid is the size of the board, but is only true for cells on the red side with food). Also, note that you can list a team's indices with getRedTeamIndices, or test membership with isOnRedTeam.

Distance Calculation: To facilitate agent development, we provide code in pacai.core.distanceCalculator to supply shortest path maze distances.

Useful Methods: To get started designing your own agent, we recommend subclassing the pacai.agents.capture.capture.CaptureAgent class. This provides access to several convenience methods. Some of these useful methods are:

- getFood
- getFoodYouAreDefending
- getOpponents
- getTeam

- getScore
- getMazeDistance
- getPreviousObservation
- getCurrentObservation

Restrictions: You are free to design any agent you want. However, you will need to respect the provided APIs if you want to participate in the tournaments. Agents which compute during the opponent's turn will be disqualified. In particular, any form of parallelism is disallowed, because we have found it very hard to ensure that no computation takes place on the opponent's turn.

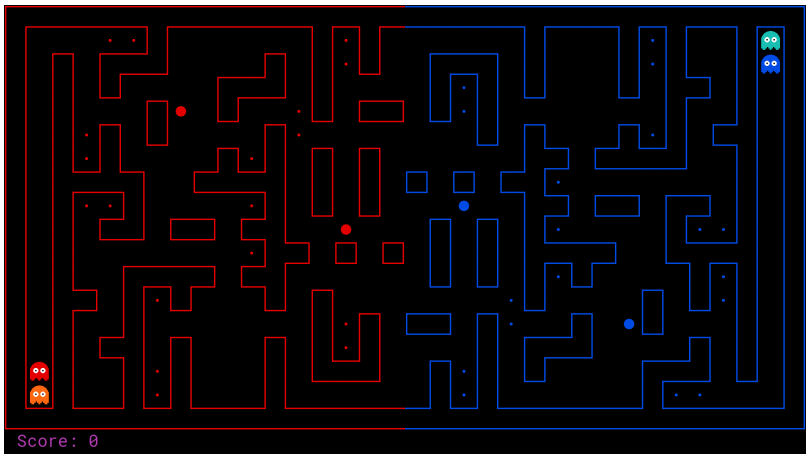
Warning: If one of your agents produces any stdout/stderr output during its games in the nightly tournaments, that output will be included in the contest results posted on the website. Additionally, in some cases a stack trace may be shown among this output in the event that one of your agents throws an exception. You should design your code in such a way that this does not expose any information that you wish to keep confidential.

Contest Details

Teams: We highly encourage you to work in teams of three people (no more than three).

Prizes: The performance-based portion of your grade will be based on the placement received in **the final** round-robin tournament. Placement is determined by the number of wins (if multiple teams have the same number of wins, it will be broken by the number of ties).

Extra Credit: Teams that beat the baseline agent in the mid-project checkpoint contest will receive 2 extra credit points.



Have fun!
Please bring our attention to any problems you discover.